

Tecnólogo em Análise e Desenvolvimento de Sistemas

Discente:Doris Helena Nunes dos Santos

Docente: Beatriz Conceição da Costa

PRINCÍPIOS E PADRÕES DE ARQUITETURA

Proposta e Visão do Sistema: Banco Comunitário Solidário

Proposta Arquitetural

Rio Grande – RS

2026

Proposta Arquitetural : Banco Comunitário Solidário

Parte Conceitual e Arquitetural — Banco SoliData

1. Abordagem Arquitetural Adotada

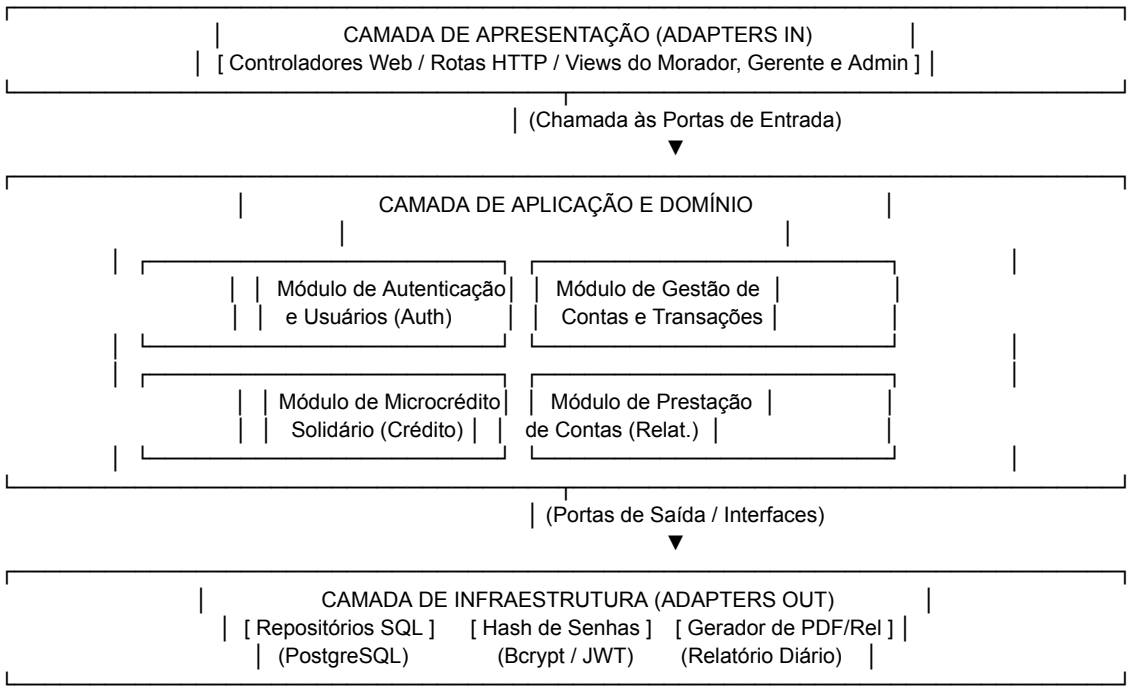
A abordagem arquitetural adotada para o Banco SoliData é a combinação entre Monólito Modular e Arquitetura Hexagonal (Ports and Adapters), apoiada pelo padrão Cliente-Servidor (Web).

Raciocínio da Combinação:

- Monólito Modular: O sistema é empacotado e implantado como uma unidade única para simplificar a implantação e a execução local durante a fase acadêmica, reduzindo a complexidade de orquestração. Internamente, o código é dividido de forma estrita em módulos funcionais (Usuários, Contas, Crédito e Relatórios).
- Arquitetura Hexagonal: Dentro de cada módulo, isolamos o *Domínio* (regras de negócio da economia solidária) dos detalhes de *Infraestrutura* (banco de dados PostgreSQL, rotas HTTP/Express e geradores de relatórios).
- Cliente-Servidor Web: A camada de apresentação web responsiva interage via HTTP/JSON com as rotas e controllers do servidor backend.

2. Organização dos Módulos e Componentes

O sistema é dividido internamente nos seguintes componentes principais:



3. Responsabilidades de Cada Parte

- Camada de Apresentação (Adapters In): Captura as requisições do usuário (Morador, Gerente e Admin), valida os parâmetros de entrada HTTP e formata as respostas visuais (HTML/JSON).
- Módulo de Autenticação e Usuários: Gerencia o controle de acesso baseado em papéis (RBAC - Cliente, Gerente, Administrador) e o ciclo de vida do usuário.
- Módulo de Gestão de Contas e Transações: Executa saques, depósitos e transferências comunitárias, garantindo a atomicidade das operações em nível de banco de dados.
- Módulo de Microcrédito Solidário: Aplica as regras de solicitação e aprovação de empréstimos sem juros para consumo e crédito produtivo.
- Módulo de Prestação de Contas: Consolida as movimentações financeiras para gerar os relatórios diários e semanais de transparência para a comunidade.
- Camada de Infraestrutura (Adapters Out): Conecta o sistema ao banco de dados relacional PostgreSQL, gerencia transações SQL atômicas, aplica criptografia de senhas e lida com a geração de arquivos.

4. Dependências e Formas de Comunicação

- Direção das Dependências: As dependências fluem estritamente de fora para dentro: a *Infraestrutura* e a *Apresentação* dependem do *Domínio*. O *Domínio* não conhece detalhes de drivers do PostgreSQL, SQL bruto nem frameworks de interface.
- Comunicação Interna: A comunicação entre módulos no Monólito Modular ocorre via chamadas de métodos síncronos em memória, através de interfaces bem definidas (Portas).
- Comunicação Externa/Cliente: O navegador web se comunica com o backend via requisições HTTP (GET, POST, PUT, DELETE) e payloads JSON.

5. Relação da Arquitetura com os ASRs e Atributos de Qualidade

Driver / Requisito (Anterior)	Decisão Arquitetural Adotada	Como a Arquitetura Garante

Atributo: Manutenibilidade (Alteração realizada em < 30 min)	Arquitetura Hexagonal + Monólito Modular	O desacoplamento via Portas e Adapters garante que alterar consultas no PostgreSQL ou modificar a interface gráfica não afete as regras de negócio centrais.
Atributo: Usabilidade (Ação concluída em < 3 cliques)	Controllers dedicados por perfil	Rotas e visões simplificadas para Morador, Gerente e Admin eliminam etapas intermediárias no fluxo de navegação.
Atributo: Integridade de Dados (Garantia de consistência ACID)	Repositórios SQL com transações no PostgreSQL	A camada de infraestrutura utiliza o suporte nativo a transações do PostgreSQL, garantindo *rollback* imediato em caso de erro na movimentação financeira.
ASR: Simplicidade de Implantação	Monólito Modular com PostgreSQL	Todo o código da aplicação roda em um único processo e se conecta ao serviço do PostgreSQL (local ou containerizado), tornando a demonstração simples e rápida.

6. Alternativa Arquitetural Considerada

- Alternativa: Arquitetura baseada em Microserviços (serviço de contas, serviço de crédito, serviço de autenticação).
- Por que foi desconsiderada: Embora ofereça escalabilidade independente, exigiria o gerenciamento de múltiplos repositórios, conteinerização de vários

serviços, orquestração de rede e tratamento de consistência eventual em um prazo de desenvolvimento extremamente curto.

7. Justificativa das Escolhas e Trade-offs

Justificativas

1. Monólito Modular: Oferece o equilíbrio ideal para a fase acadêmica: facilidade e rapidez de implementação sem abrir mão da organização interna do código.
2. Arquitetura Hexagonal: Prepara o projeto para o futuro. Caso seja necessário substituir o PostgreSQL por outro serviço de banco ou adicionar um aplicativo mobile nativo posteriormente, a regra de negócio do Banco SoliData permanece intacta.
3. Uso do PostgreSQL: Garante robustez comercial, alto desempenho e conformidade estrita com as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade), essenciais para o controle bancário de saldos e transações.

Trade-offs (Prós e Contras)

- Vantagens (Ganhos):
  - Baixa complexidade de infraestrutura e teste.
  - Alta consistência financeira com suporte transacional nativo do PostgreSQL.
  - Alta coesão interna dos módulos de negócio.
- Desvantagens (Concessões):
  - Exige a execução de um servidor de banco de dados (PostgreSQL local ou via Docker) durante os testes.
  - Toda a aplicação precisa ser implantada novamente caso um único módulo seja modificado.

8. Diagrama Arquitetural da Proposta (Código Mermaid)

```
graph TB
   subgraph Cliente [Camada de Apresentação / Interface]
      WebClient[Navegador Web - Responsivo]
   end

   subgraph Servidor [Monólito Modular - Backend Banco SoliData]
      subgraph AdaptersIn [Adaptação de Entrada]
         HTTPControllers[Controladores HTTP / Express / Rotas]
      end

      subgraph CoreDomain [Núcleo do Domínio & Regras de Negócio]
         AuthModule[Módulo de Usuários & Auth]
         AccountModule[Módulo de Contas & Transações]
```

```
        CreditModule[Módulo de Microcrédito]
        ReportModule[Módulo de Prestação de Contas]
    end

    subgraph AdaptersOut [Adaptação de Saída / Infraestrutura]
        RepoImpl[Repositórios SQL Data Access]
        PDFGen[Gerador de Comprovantes/PDF]
    end
end

subgraph Persistencia [Camada de Dados]
    DB[(Banco de Dados: PostgreSQL)]
end

%% Comunicação
WebClient -->|HTTP / JSON| HTTPControllers
HTTPControllers --> CoreDomain
CoreDomain --> RepoImpl
CoreDomain --> PDFGen
RepoImpl -->|SQL Transacional / ACID| DB
```